

Programming as Theory Building - Peter Naur (1985)

1. Introduction (Introducción)

Palabras clave:

- Programación
- Intuición
- Teoría de la programación
- Producción de software
- Desarrollo de software
- Modificación de programas
- Ejecución inesperada
- Construcción de teoría
- Dificultades en programación
- Visión de Construcción de Teoría

Resumen:

Este texto propone una perspectiva alternativa sobre la programación, sugiriendo que más que la simple producción de código y documentación, programar implica la construcción de una intuición o teoría sobre el problema abordado. Se cuestiona la visión tradicional que ve la programación solo como la producción de software, argumentando que esta no logra explicar adecuadamente los desafíos reales que surgen en el desarrollo y mantenimiento de sistemas. La falta de una comprensión adecuada de la naturaleza de la programación puede generar conflictos y frustraciones en el proceso de desarrollo. Como alternativa, se introduce la **Visión de Construcción de Teoría**, la cual será desarrollada en el texto.

Conceptos clave:

1. **Programación como construcción de conocimiento:** No se trata solo de escribir código, sino de desarrollar una teoría sobre el problema que se intenta resolver.
 2. **Limitaciones de la visión tradicional:** La programación vista solo como producción de software ignora las dificultades reales en el mantenimiento y evolución de sistemas.
 3. **Ejecuciones inesperadas:** Los problemas en el software suelen surgir de interacciones imprevistas que desafían una visión simplista de la programación.
 4. **Modificación de programas:** Cada cambio en el código requiere una comprensión profunda del diseño, que no siempre es fácil de transmitir mediante documentación.
 5. **Impacto de una comprensión errónea:** Si no se entiende correctamente qué implica programar, los esfuerzos por mejorar los procesos pueden ser ineficaces o generar frustración.
-

2. Programming and the Programmers' Knowledge (La programación y el conocimiento del programador)

Palabras clave:

- Programación
- Conocimiento del programador
- Diseño de software
- Implementación
- Documentación
- Compilador
- Diagnóstico de fallas
- Adaptación y mantenimiento
- Transmisión de conocimiento
- Teoría del diseño

Resumen:

La programación no se reduce a escribir código, sino que implica la construcción de conocimiento por parte de los programadores. La documentación, aunque útil, no siempre es suficiente para transmitir la comprensión profunda del diseño de un sistema. Dos casos ilustran esta idea:

1. **Desarrollo de compiladores:** Un grupo intentó ampliar un compilador utilizando documentación y asesoría externa. Sin embargo, solo los creadores originales pudieron transmitir la lógica interna del diseño.
2. **Sistemas de monitoreo industrial:** Programadores experimentados pueden diagnosticar fallas basándose en su conocimiento inmediato del sistema, mientras que otros grupos con documentación detallada enfrentan dificultades.

Esto demuestra que el mantenimiento y evolución de sistemas complejos dependen de un conocimiento tácito y continuo de los programadores involucrados desde el inicio.

Conceptos clave:

1. **Conocimiento tácito:** Los programadores acumulan una comprensión profunda del sistema que no siempre puede expresarse en documentación.
2. **Limitaciones de la documentación:** No garantiza la transferencia completa del conocimiento sobre el diseño y funcionamiento del software.
3. **Importancia de la experiencia:** La interacción directa y el tiempo de trabajo continuo con un sistema permiten un entendimiento que supera lo escrito.
4. **Evolución del software:** Los sistemas cambian con el tiempo, y el conocimiento de los programadores que los desarrollaron es crucial para mantener su eficacia.
5. **Impacto de la falta de transmisión efectiva:** Sin una adecuada transferencia de conocimiento, los sistemas tienden a degradarse con el tiempo.

3. Ryle's notion of Theory (La noción de la teoría según Ryle)

Palabras Claves:

- **Gilbert Ryle**
- **Teoría según Ryle**
- **Actividad intelectual vs. actividad inteligente**
- **Conocimiento y teoría**
- **Justificación y explicaciones**
- **Reglas y regresión infinita**
- **Similitudes en el conocimiento**
- **Aplicación de teorías**
- **Popper y el Mundo 3**
- **Thomas Kuhn y la aplicación de teorías**

Resumen:

Según **Gilbert Ryle**, el conocimiento de los programadores debe entenderse como una **teoría**, ya que no solo permite realizar tareas inteligentes sino también explicarlas y justificarlas. La actividad intelectual trasciende la mera inteligencia porque implica la construcción y apropiación de una teoría que permite analizar y modificar el conocimiento adquirido.

Ryle distingue entre:

- **Actividad inteligente:** Capacidad de hacer algo correctamente sin necesariamente poder explicarlo.
- **Actividad intelectual:** Capacidad de realizar algo inteligentemente y, además, justificarlo, discutirlo y aplicarlo en nuevos contextos.

Una teoría no es solo un conjunto de reglas, ya que seguir reglas en sí mismo puede hacerse de manera más o menos inteligente. Además, si la inteligencia dependiera solo de reglas, llevaría a una **regresión infinita** sobre cómo seguirlas.

El conocimiento teórico también implica la capacidad de **reconocer similitudes** en el mundo real, lo que permite aplicar conceptos a nuevos problemas. Este tipo de conocimiento no puede reducirse a reglas estrictas, de la misma manera que no se pueden formular criterios precisos para identificar caras, melodías o sabores.

Conceptos Claves:

- **Teoría como conocimiento aplicado:** No solo es saber algo, sino poder justificarlo y aplicarlo en nuevos contextos.
 - **Diferencia entre actividad intelectual e inteligencia:** La inteligencia es hacer algo bien; la actividad intelectual es comprenderlo y explicarlo.
 - **Problema de la regresión infinita:** Si la inteligencia dependiera de reglas, necesitaríamos reglas para seguir reglas, generando un problema infinito.
 - **Similitudes en el conocimiento:** Una teoría no solo establece principios abstractos, sino que permite reconocer situaciones similares y aplicar el conocimiento en ellas.
 - **Ejemplo de la mecánica de Newton:** No basta con conocer la ecuación $F=ma$; es necesario comprender cómo se aplica a fenómenos como los péndulos y los planetas.
 - **Mundo 3 de Popper:** Las teorías forman parte de un conocimiento objetivo e independiente de la mente individual.
 - **Aplicabilidad de la teoría:** No se limita a disciplinas científicas, sino que también se aplica en situaciones cotidianas, como organizar muebles o planificar un viaje.
-

4. The Theory To Be Built by the Programmer (La teoría a ser construida por el programador)

Palabras Claves:

- **Visión de Construcción de Teoría (VCT)**
- **Teoría del Programador**
- **Conocimiento del Programador**
- **Mundo Real y su Representación**
- **Justificación del Programa**
- **Modificación de Programas**
- **Similitudes entre Demandas y Código**
- **Explicación y Definición del Código**
- **Principios y Reglas de Diseño**

- **Percepción de la Similitud**

Resumen:

La Visión de Construcción de Teoría (VCT) sostiene que el aspecto más importante de la programación no es la escritura del código ni la documentación, sino la teoría que el programador construye sobre cómo el programa representa aspectos del mundo real. Según esta visión, el conocimiento del programador es superior a la documentación en tres aspectos esenciales:

1. **Relación con el mundo real:** El programador debe entender y explicar cómo el programa modela los problemas del mundo real y qué aspectos son relevantes.
2. **Justificación del código:** Cada parte del programa debe poder justificarse con base en principios, reglas de diseño o el conocimiento intuitivo del programador.
3. **Facilidad de modificación:** Un programador con la teoría del programa puede responder mejor a modificaciones, identificando similitudes entre las nuevas demandas y las funcionalidades existentes.

El VCT argumenta que los problemas de la programación no pueden reducirse a simples reglas, sino que dependen del conocimiento del programador sobre el mundo y la solución implementada.

Conceptos Claves:

- **Teoría del Programador:** La comprensión profunda que el programador tiene sobre cómo su código modela y resuelve problemas del mundo real.
- **Relación con el Mundo Real:** Todo programa es una representación de un problema en el mundo, y el programador debe decidir qué aspectos son relevantes.
- **Justificación del Código:** No basta con escribir código funcional, sino que debe haber una razón detrás de cada decisión de diseño.
- **Modificaciones y Similitud:** La facilidad para modificar un programa depende de la capacidad del programador para identificar similitudes entre los nuevos requerimientos y lo ya implementado.
- **Limitaciones de la Documentación:** La teoría que el programador construye trasciende lo que se puede escribir en la documentación, ya que

incluye conocimiento intuitivo y experiencia.

5. Problems and Costs of Program Modifications (Problemas y costos de modificar un programa)

Palabras clave

- **Modificación de programas**
- **Costo de mantenimiento**
- **Flexibilidad del software**
- **Construcción de teoría**
- **Calidad del código**
- **Similitud entre requerimientos**
- **Deterioro del software**
- **Estructura y simpleza**

Resumen

El texto analiza los problemas y costos de modificar programas dentro del marco de la Visión de Construcción de Teoría (VCT). Se argumenta que la modificación del software es inevitable, pero no siempre es económica ni viable. Se desestima la idea de que modificar un programa sea barato simplemente porque es texto editable, ya que la programación no se reduce a la manipulación de texto sino a la construcción de una teoría. También se discute la flexibilidad en los programas, señalando que agregar flexibilidad innecesaria tiene costos elevados. Finalmente, se resalta la importancia de que las modificaciones sean realizadas por programadores con conocimiento profundo de la teoría subyacente del software, pues cambios mal fundamentados pueden degradar la calidad del código a largo plazo.

Conceptos clave

1. **Inevitabilidad de la modificación:** Todo software en producción será modificado debido a nuevas necesidades o mejoras.
2. **Costo de modificar vs. reconstruir:** A diferencia de otras construcciones humanas (autos, edificios), el software no siempre es barato de modificar, y en algunos casos podría ser mejor rehacerlo desde cero.

3. **Flexibilidad y sus costos:** Diseñar programas flexibles para futuras modificaciones puede ser costoso e innecesario si no hay una clara justificación.
 4. **Importancia de la teoría:** Para modificar correctamente un programa, se debe entender su teoría subyacente. Modificaciones sin este entendimiento pueden degradar el código y convertirlo en un conjunto de parches incoherentes.
 5. **Diferencias entre modificaciones:** No todas las modificaciones son iguales; algunas extienden la teoría del programa de manera natural, mientras que otras la contradicen y generan código de baja calidad.
 6. **Sostenibilidad del software:** Para mantener la calidad de un programa a largo plazo, las modificaciones deben estar alineadas con la teoría original.
-

6. Program Life, Death and Revival (La Vida, Muerte y Resurrección de un Programa)

Palabras Claves

- **Visión de Construcción de Teoría (VCT)**
- **Teoría de un programa**
- **Vida, muerte y resurrección de un programa**
- **Equipo de programadores**
- **Transferencia de conocimiento**
- **Aprendizaje práctico**
- **Documentación y teoría**
- **Mantenimiento de software**
- **Evolución de equipos de desarrollo**
- **Reescritura vs. mantenimiento**

Resumen

La Visión de Construcción de Teoría (VCT) de la programación establece que un programa no puede separarse de la teoría que lo sustenta, la cual reside en los programadores. Bajo esta perspectiva, un programa atraviesa tres etapas:

vida, cuando su equipo original sigue a cargo; **muerte**, cuando se disuelve el equipo y no es posible modificar el programa inteligentemente; y **resurrección**, cuando un nuevo equipo intenta reconstruir la teoría original.

La transmisión de la teoría no se logra solo con documentación, sino con aprendizaje práctico bajo la guía de programadores con conocimiento previo. La VCT sugiere que intentar revivir un programa es ineficiente y costoso, y que, en muchos casos, es preferible desarrollar una nueva solución desde cero. Además, la evolución continua de los equipos de desarrollo puede generar cambios en la teoría del programa, lo que dificulta su mantenimiento a largo plazo.

Conceptos Importantes

- **Teoría de un programa:** Es el conocimiento implícito en los programadores sobre cómo y por qué funciona un programa, lo que no puede expresarse completamente en documentación.
- **Vida del programa:** Período en el que el equipo original está activo y puede realizar modificaciones.
- **Muerte del programa:** Ocurre cuando el equipo original desaparece y no hay suficiente conocimiento para realizar cambios significativos.
- **Resurrección del programa:** Intento de un nuevo equipo de reconstruir la teoría original, proceso que suele ser costoso y generar discrepancias con el programa inicial.
- **Limitaciones de la documentación:** La documentación por sí sola no puede transmitir la teoría de un programa; se requiere contacto directo con programadores experimentados.
- **Dificultad de mantenimiento a largo plazo:** A medida que los equipos evolucionan, la teoría cambia, lo que puede generar inconsistencias en el desarrollo del software.
- **Reescritura como alternativa:** En muchos casos, es preferible desarrollar una nueva solución en lugar de intentar revivir un programa muerto.

7. Method and Theory Building

Palabras clave

- **Visión de Construcción de Teoría (VCT)**
- **Método de programación**
- **Teoría subyacente**
- **Desarrollo de software**
- **Método científico**
- **Estrategias flexibles**
- **Experimentación controlada**
- **Éxito de los métodos**
- **Educación de programadores**
- **Estructuración de sistemas**

Resumen

La **Visión de Construcción de Teoría (VCT)** se diferencia de los métodos tradicionales de programación al considerar que el proceso de desarrollo de software no debe estar regido por una secuencia rígida de acciones ni por métodos estandarizados.

Se argumenta que la idea de un

método de programación universalmente correcto es errónea, ya que el desarrollo de software no puede seguir un procedimiento formal fijo. Se cuestiona la existencia de un **método científico único**, destacando que incluso en la ciencia no se sigue un conjunto de instrucciones estrictas, sino más bien un conjunto de estrategias flexibles.

Desde la perspectiva de la VCT, la importancia de los métodos radica en su papel educativo, ayudando a los programadores a familiarizarse con modelos de solución, técnicas de verificación y principios de estructuración de sistemas complejos. Sin embargo, la elección de qué técnicas emplear y en qué orden debe depender del programador y del problema a resolver, en lugar de ser impuesta por un procedimiento rígido.

Conceptos clave

1. **Métodos de programación:** Conjunto de reglas que dictan cómo desarrollar software, en qué orden y con qué herramientas.
2. **Diferencia entre VCT y métodos tradicionales:** La VCT enfatiza la teoría personal del programador sobre el sistema, en lugar de seguir pasos

predefinidos.

3. **Crítica a la idea de un método único:** No hay evidencia concluyente de que ciertos métodos sean más efectivos; su éxito reportado es anecdótico.
 4. **Falsa analogía con la ciencia:** No existe un único método científico; los científicos usan estrategias flexibles y adaptables.
 5. **Importancia de la educación:** En lugar de imponer reglas fijas, la enseñanza debe enfocarse en modelos de solución y principios de estructuración de sistemas.
-

8. Programmers' Status and the Theory Building View (El Estatus del Programador y la Visión de la Construcción de la Teoría)

Palabras Claves

- Visión de Construcción de Teoría (VCT)
- Estatus del programador
- Producción industrial vs. creación intelectual
- Teoría subyacente en programación
- Formación y educación del programador
- Reemplazabilidad del programador

Resumen

En este capítulo se contrasta la **Visión de Construcción de Teoría (VCT)** con la visión predominante sobre la programación. La perspectiva tradicional trata a *los programadores como piezas reemplazables en una producción industrial*, siguiendo reglas y procedimientos estrictos. En cambio, la **VCT** considera que el principal producto de la programación es la teoría que los programadores mantienen en su mente, lo que implica que no pueden ser tratados como meros operarios. En esta visión, *los programadores deben recibir un estatus similar al de otros profesionales* como ingenieros o abogados, con una educación enfocada en la formación de teorías más que en la simple aplicación de reglas.

Conceptos Claves

1. Diferencia entre la VCT y la visión tradicional:

- La visión tradicional ve la programación como una actividad mecanizada e industrial.
- La VCT enfatiza la construcción de teorías y la importancia del programador como un profesional intelectual.

2. El estatus del programador:

- Según la VCT, los programadores no son reemplazables y deben ser reconocidos como profesionales de alto nivel.
- Deberían ocupar un lugar similar al de ingenieros y abogados, con mayor responsabilidad en sus proyectos.

3. Impacto en la educación:

- La educación en programación debe enfocarse en la formación de teorías y en la resolución de problemas en entornos activos y supervisados.
- La enseñanza basada solo en reglas y procedimientos es insuficiente para el desarrollo del talento en programación.

4. Crítica al enfoque mecanicista de la programación:

- La idea de que los programadores deben actuar como máquinas y seguir reglas estrictas es vista como un error.
 - La programación implica creatividad y construcción intelectual, no solo ejecución de instrucciones.
-